



Logistic Regression

Rasha Obeidat, PhD
AI342

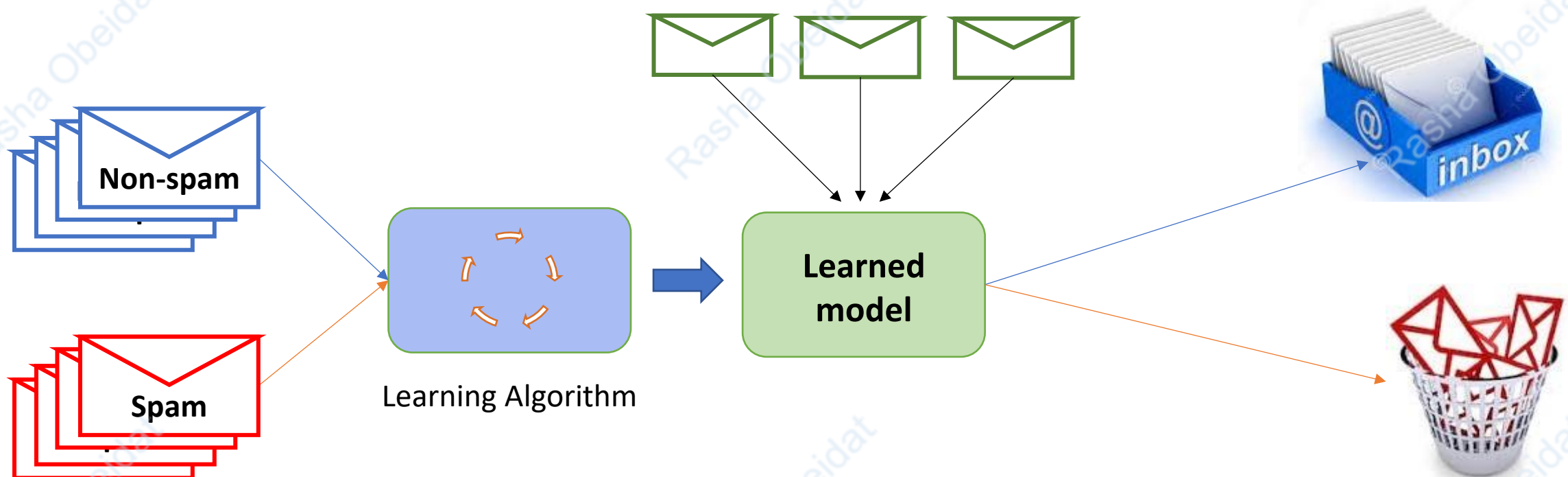
Classification

- It is assigning an instance into one or more of a predefined set of class labels.



Classification: Examples

- Spam detection



Classification: Examples

- Credit approval

Age	36
Annual Income	35,000
Available debit	10,000
Year in job	3
Credit scores	715

Applicant information



Classification: Examples

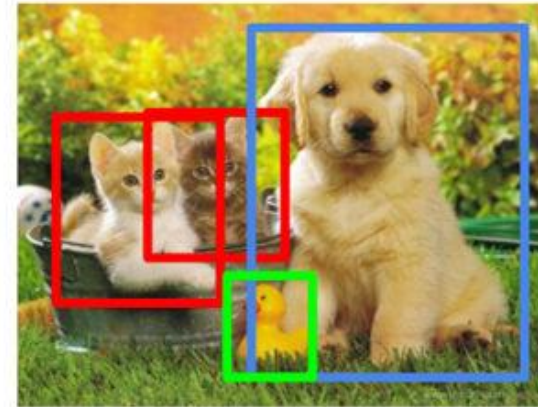
- Object Detection: **detecting** instances of **objects** from a particular class in an image.

Classification



CAT

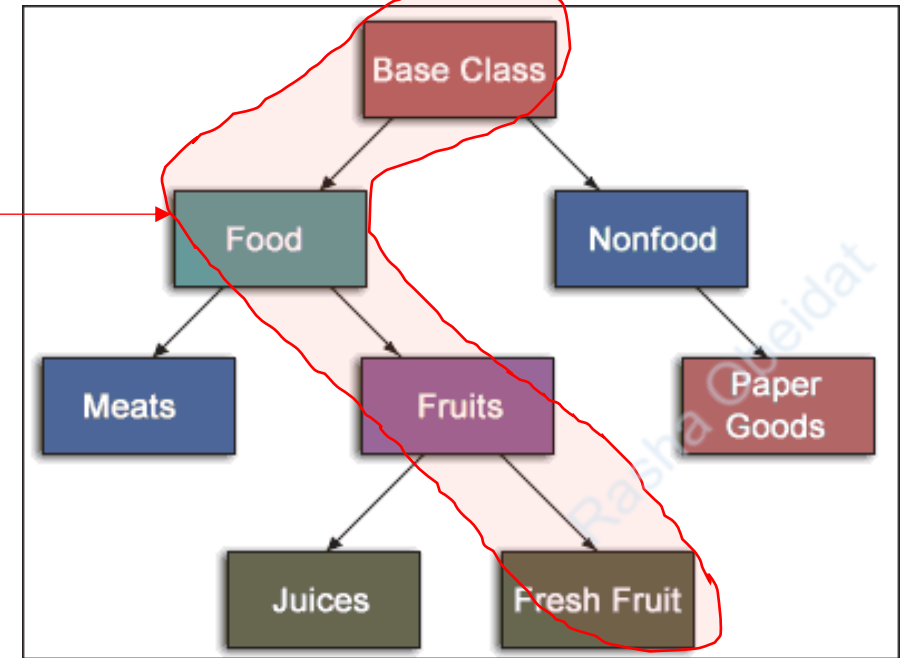
Object Detection



CAT, DOG, DUCK

Hierarchical vs. Flat Classification

- **Hierarchical Classification:** assigning an instance to one or more class labels corresponds to selecting a *path in a type hierarchy*.
 - Classes are organized in levels, where child classes represent more specific types of their parent classes.
- **Flat Classification:** In flat classification, all classes exist at the *same level*, and there is no hierarchical relationship between them. The model simply chooses one (or more) labels from a single set of *independent* classes.



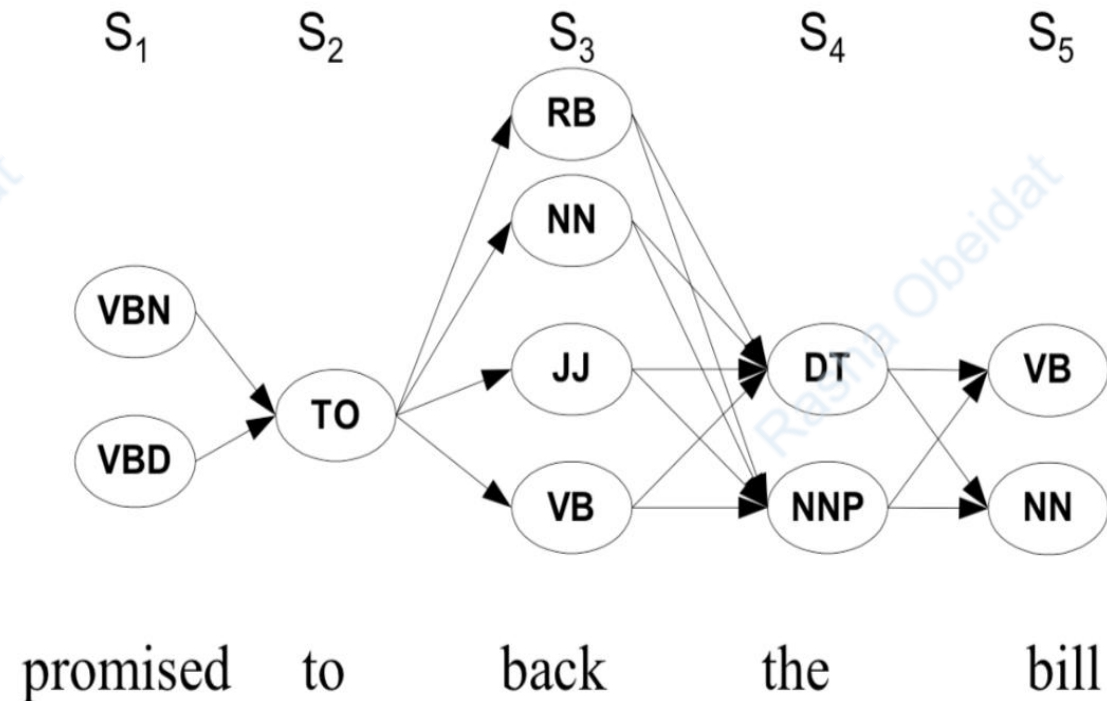
Hierarchical Classification

Structured Prediction

- Structured prediction is a type of classification problem in which the **output variables** are **interdependent** or **constrained**.
- Instead of predicting a single independent label, the model predicts a structured output where labels influence each other.
- **Example**

Part-of-Speech (POS) Tagging

The POS tag assigned to a word depends on the tags of neighboring words in the sentence.



([Mayank Singh NLP 2019](#))

Tag	Description	Example	Tag	Description	Example
CC	coord. conjunction	<i>and, or</i>	RB	adverb	<i>extremely</i>
CD	cardinal number	<i>one, two</i>	RBR	adverb, comparative	<i>never</i>
DT	determiner	<i>a, the</i>	RBS	adverb, superlative	<i>fastest</i>
EX	existential there	<i>there</i>	RP	particle	<i>up, off</i>
FW	foreign word	<i>noire</i>	SYM	symbol	<i>+, %</i>
IN	preposition or sub-conjunction	<i>of, in</i>	TO	“to”	<i>to</i>
JJ	adjective	<i>small</i>	UH	interjection	<i>oops, oh</i>
JJR	adject., comparative	<i>smaller</i>	VB	verb, base form	<i>fly</i>
JJS	adject., superlative	<i>smallest</i>	VBD	verb, past tense	<i>flew</i>
LS	list item marker	<i>1, one</i>	VBG	verb, gerund	<i>flying</i>
MD	modal	<i>can, could</i>	VBN	verb, past participle	<i>flown</i>
NN	noun, singular or mass	<i>dog</i>	VBP	verb, non-3sg pres	<i>fly</i>
NNS	noun, plural	<i>dogs</i>	VBZ	verb, 3sg pres	<i>flies</i>
NNP	proper noun, sing.	<i>London</i>	WDT	wh-determiner	<i>which, that</i>
NNPS	proper noun, plural	<i>Azores</i>	WP	wh-pronoun	<i>who, what</i>
PDT	predeterminer	<i>both, lot of</i>	WP\$	possessive wh-	<i>whose</i>
POS	possessive ending	<i>'s</i>	WRB	wh-adverb	<i>where, how</i>
PRP	personal pronoun	<i>he, she</i>			

Classification Problem Formalization

- Turn every example into a vector $x_i \in \mathbb{R}^d$.
- Training set consists of a set of example-label pairs: $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)$.
- $y^i \in Y$ is its corresponding output of the x^i . Y is a predefined set of class labels.
- We assume that there is some underlying target function $f(x)$ that maps x to y .
- Our goal is to find an approximation $g(x)$ of the function $f(x)$.

Classification Problem Formalization

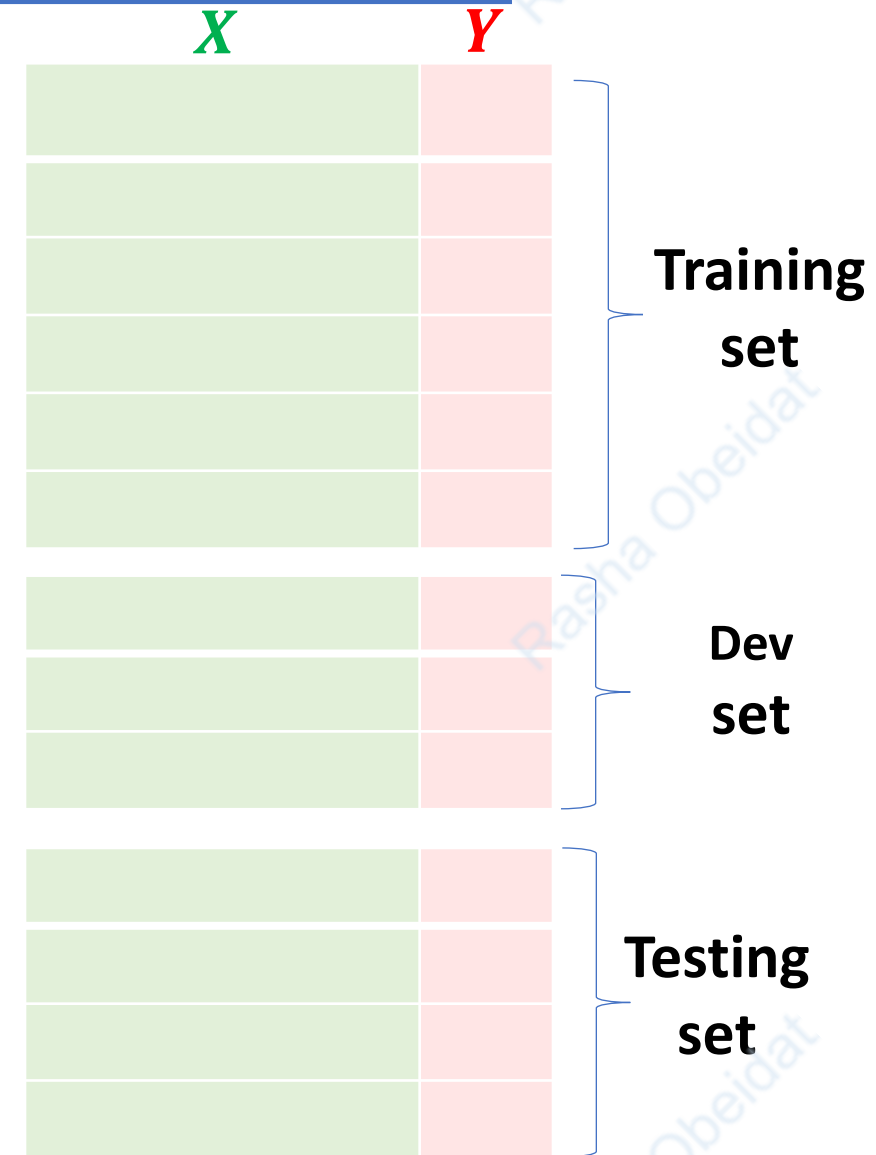
The components of the spam detection problem:

- Input: $x \in X$. (the contents of an emails)
- Output: $y \in Y$. (spam or non-spam)
- Data: $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)$. (emails already classified into spam/non-spam.)
- Target function : $f(x): X \rightarrow Y$ (the optimal spam detector)
- Hypothesis: $g(x): X \rightarrow Y$ (the spam detector to be learned from the data, which an approximation of $f(x)$)

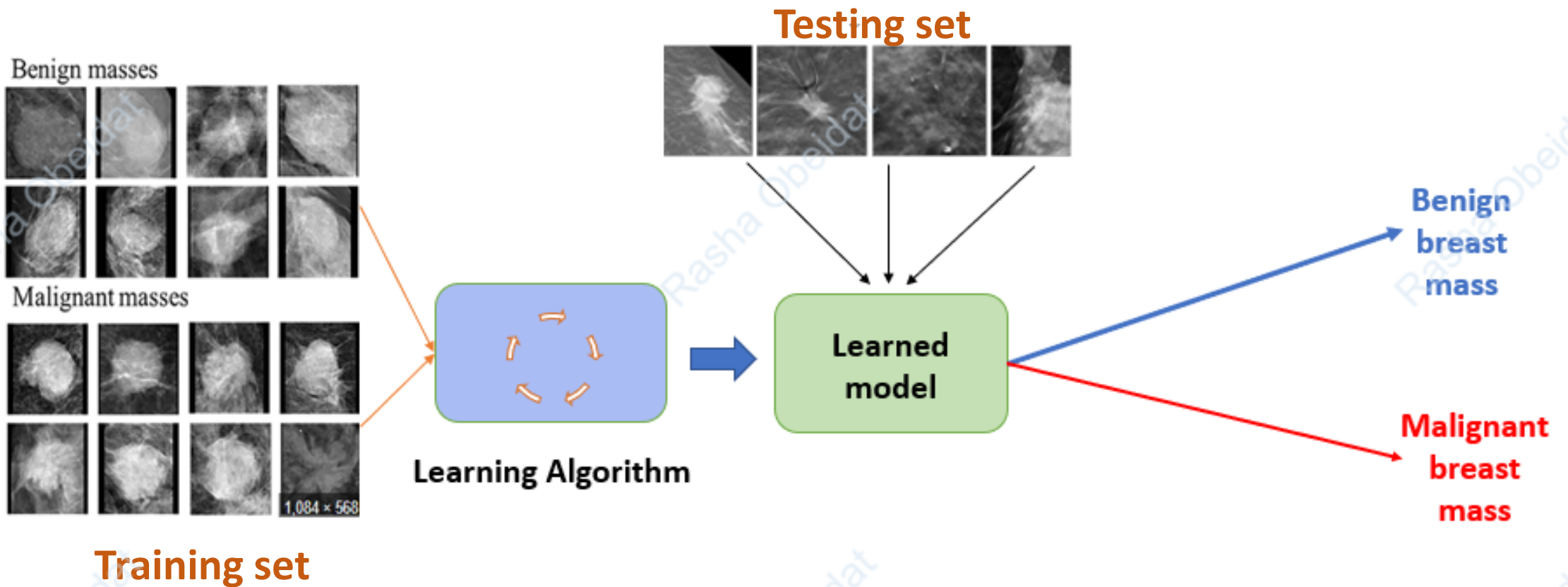
Supervised Learning: splitting Up Data

Data Splitting:

- **Training Set:** used to train the model on known X - Y pairs.
- **Development Set (validation set):** used to adjust hyperparameters and improve model during development.
- **Test Set:**
 - Used to evaluate a model's performance on *unseen* data by comparing its *predictions* ($\hat{Y}$) against *true labels* (Y).
 - Evaluation metrics (e.g., *accuracy*)
 - Quantify how well the model generalizes to new data



Supervised Learning: splitting Up Data



Probabilistic Classifier

- That classifier that to every class, i.e. compute $p(y|x) \forall y \in Y$.
computes the probability that the instance belongs
- E.g. compute $p(y = \textit{spam} | x)$ and $p(y = \textit{non spam} | x)$.
- $P(y|x)$: the *posterior* probability of y given x .
- The prediction is performed using the **decision theory**.

Decision theory

- Design classifiers to recommend decisions that minimize some total expected "risk".
- **Criterion 1**: Make a decision that minimizing the misclassification rate (the simplest risk in which costs are equal).

$$y^* = \operatorname{argmax}_y p(y|x) .$$

- E.g., $p(y = \textit{spam}|x) = 0.7 \Rightarrow$ assign **spam** to x .

Decision theory

What if $p(y = \text{spam}|x) = 0.51$ and $p(y = \text{non spam}|x) = 0.49$, dose it make sense to make prediction according to criteria 1?

- **Criterion 2:** Make a decision that minimizes the overall expected cost (loss).
 - Typically, the risk includes the cost associated with different decisions.
 - Given a cost/loss matrix $L(y, \hat{y})$, find the cost of different types of mistakes, and the assign x to y with minimum expected cost:
$$y^* = \operatorname{argmin}_{\hat{y}} \sum_{y \in \{s, ns\}} L(y, \hat{y}) p(y|x) .$$
 - Where y is the actual class and $\hat{y}$ is the predicted.

Example

- If a spam detector give a posterior probability of $p(y = \text{spam}|x) = 0.7$ of an incoming email x , what is the expected cost of predicting spam and non-spam given the following cost matrix?

predicted → Actual ↓	spam	Non-spam
spam	0 (TP)	1 (FN)
Non-spam	10 (FP)	0 (TN)

- The expected cost of predicting **spam** = $\sum_{y \in \{\text{s}, \text{ns}\}} L(y, \text{s}) p(y|x)$
 $= L(\text{s}, \text{s}) \times p(y = \text{s}|x) + L(\text{ns}, \text{s}) \times p(y = \text{ns}|x) = 0 \times .7 + 10 \times .3 = 3$

Example: continue...

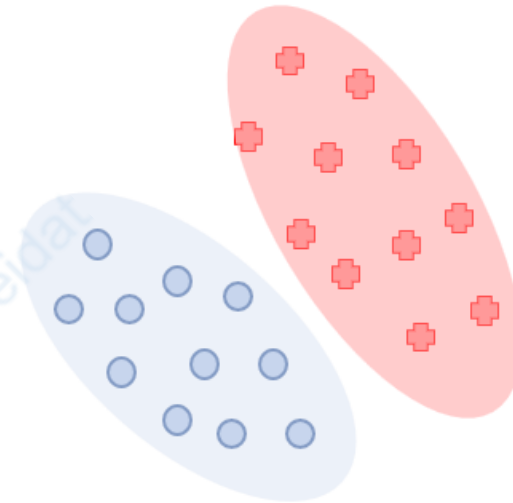
predicted → Actual ↓	spam	Non-spam
spam	0 (TP)	1 (FN)
Non-spam	10 (FP)	0 (TN)

- The expected cost of predicting **non-spam** = $\sum_{y \in \{s, ns\}} L(y, \text{ns}) p(y|x)$
 $= L(s, \text{ns}) \times p(y = s|x) + L(\text{ns}, \text{ns}) \times p(y = \text{ns}|x) = 1 \times .7 + 0 \times .3 = 0.7$

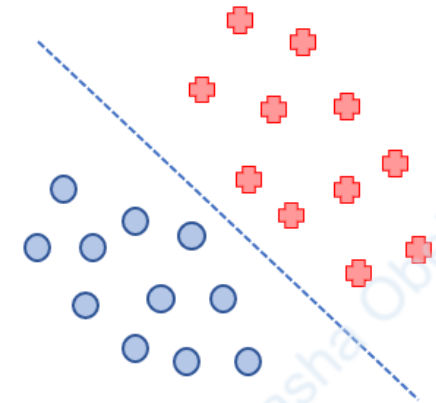
So, the email is assigned the class **non-spam** as it has the minimum expected loss.

Types of Classifiers

- **Generative:**
 - Model the distribution of every class.
- **Discriminative:**
 - Model the decision boundary between the classes.



Generative classifiers



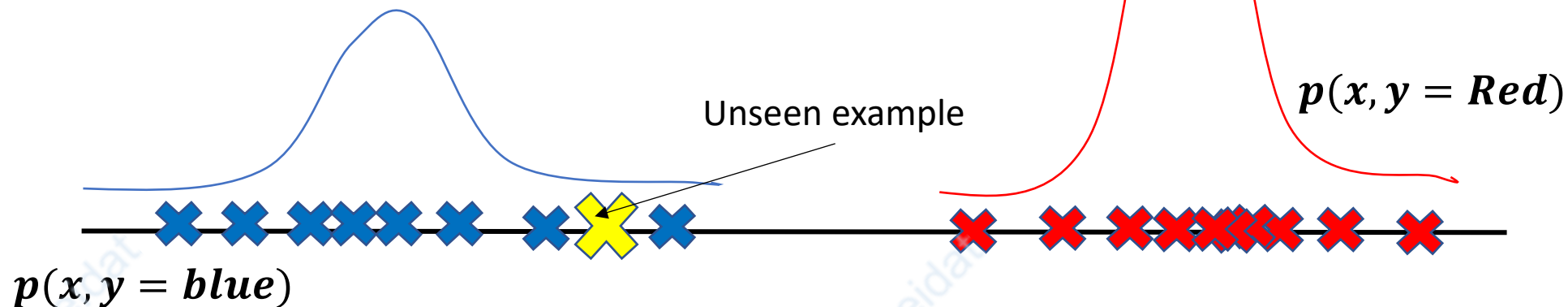
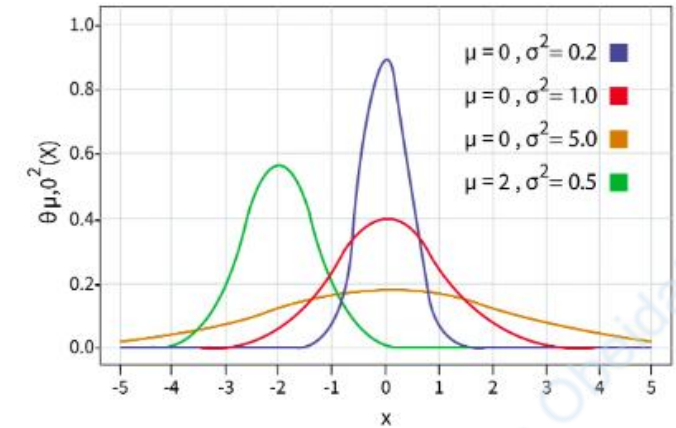
Discriminative classifiers

Generative Classifiers

- Describe how data is **generated** using probabilistic models.
- learn the distribution of data in every class, i.e., model the joint distribution $p(x, y)$.
- $p(x, y) = p(x|y) \times p(y)$
- Learn $p(y)$ and $p(x|y)$ from the data.
- Can be used to create synthetic examples that resemble the training data.
- Compute $p(y|x)$ using Bayes rule:
- $$p(y|x) = \frac{p(x|y)p(y)}{p(x)} = \frac{p(x|y)p(y)}{\sum_y p(x|y)p(y)}$$

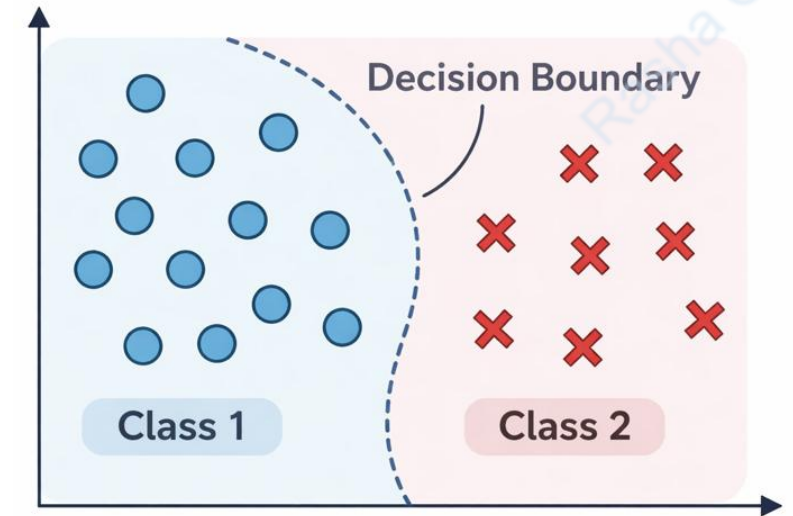
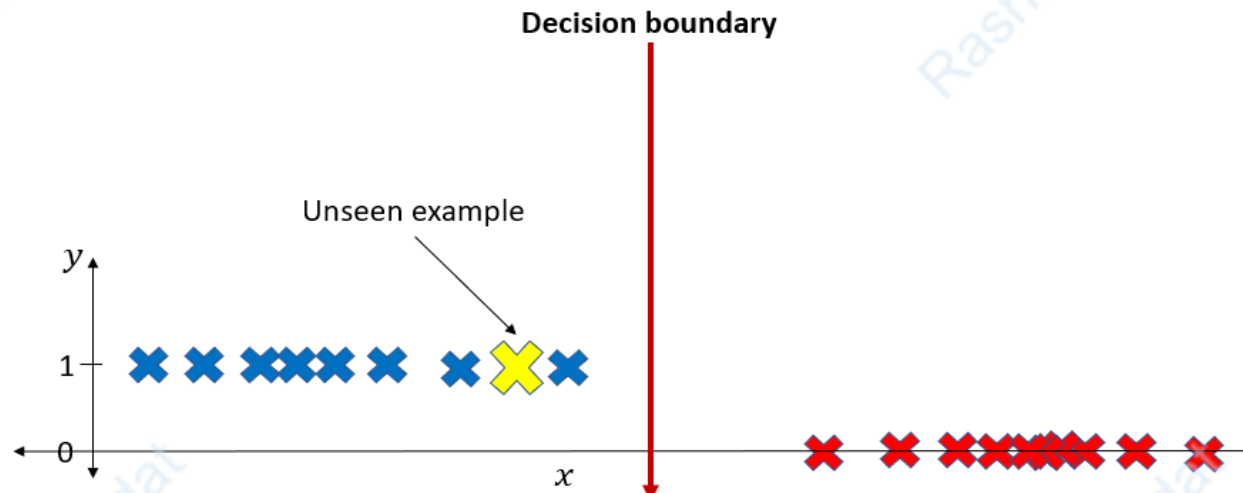
Generative Classifiers

- Compute $p(y|x)$ using Bayes rule:
- $$p(y|x) = \frac{p(x|y)p(y)}{p(x)} = \frac{p(x|y)p(y)}{\sum_y p(x|y)p(y)}$$
- Naïve Byes classifier and mixture of Gaussian are examples.



Discriminative Classifiers

- Model the decision boundary between the classes.
- Don't care how the data is generated and model the conditional probability $p(y|x)$ directly.
- Logistic regression and Neural Networks are examples.
- Work better in practice.



Cons and pros

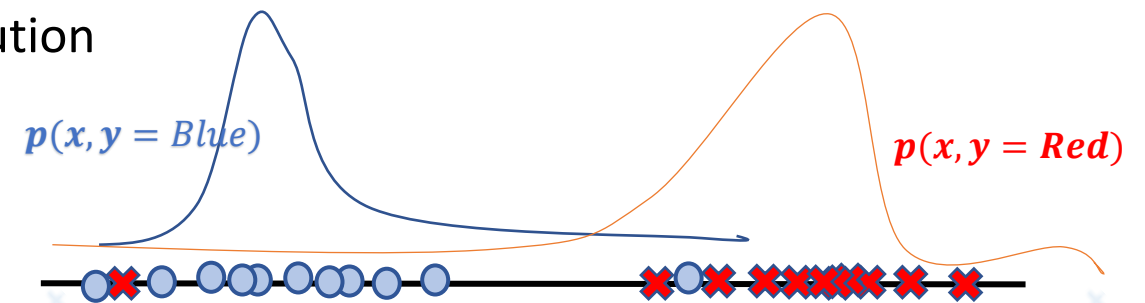
• Generative model

□ Pros

- Less prone to overfitting on small datasets.
- Have broader capabilities than only supervised learning, including generating synthetic data and learning the data distribution.

□ Cons

- Assume an underlying data distribution.
- Require sufficient data to learn the distribution reliably.
- Computationally expensive.
- Sensitive to outliers that may distort the estimated distribution.



Cons and pros

- **Discriminative models**

- **Pros**

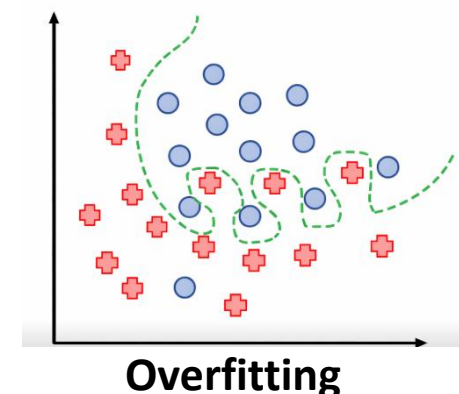
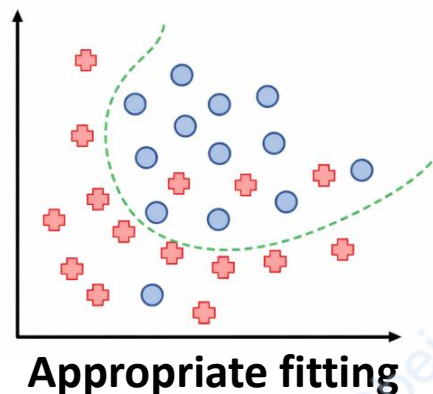
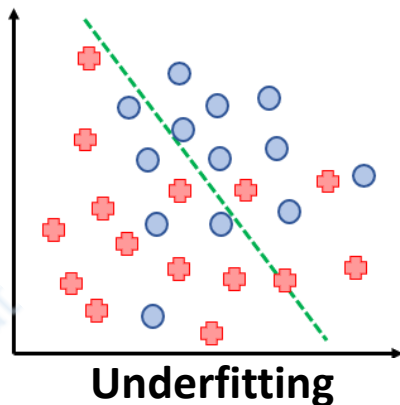
- Do not assume a probability distribution for the data.
 - Computationally cheaper.
 - Often work better in practice.

- **Cons**

- Prone to overfitting on small datasets.
 - Limited to optimizing predictive accuracy for labeled data tasks (supervised learning) and cannot be directly used for data generation.
 - May suffer more from class imbalance compared to generative models.

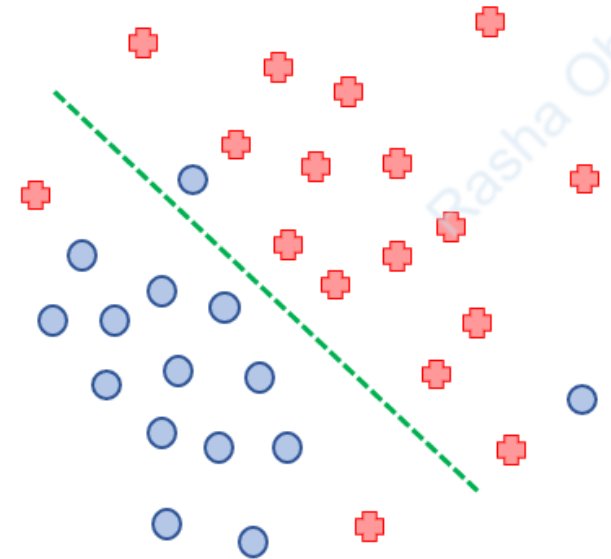
Generalization in Machine Learning

- To learn reliably from data, an ML algorithm should learn general concepts from *specific examples* and apply the learned knowledge to examples not seen before (*inductive learning*).
- **Overfitting** happens when a model closely fits the limited training data available, i.e., it learns the details and noise in the training data to the extent that it negatively impacts the model's ability to make correct predictions on new data.
- **Underfitting** happens when the model fails to adequately model the training data and therefore cannot generalize well to new data.



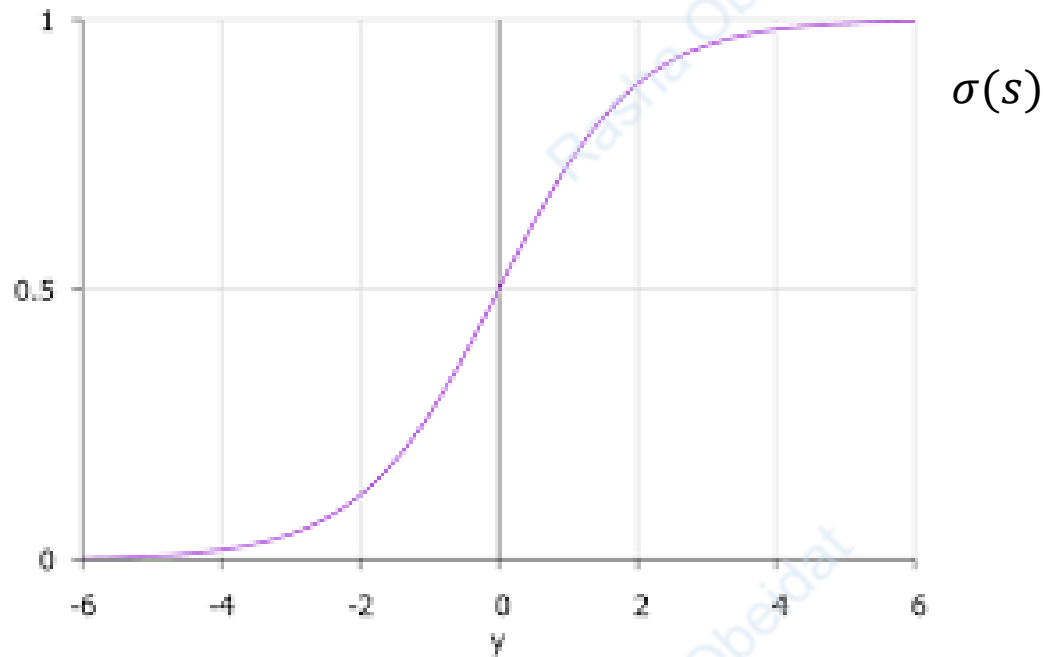
Linear classifiers

- Learn a linear line (or hyperplane in high dimension space) to separate positive from negative data point.
- Simple
- Need less data to learn reliably as they have less parameters to learn.
- Overfit less to noisy data.



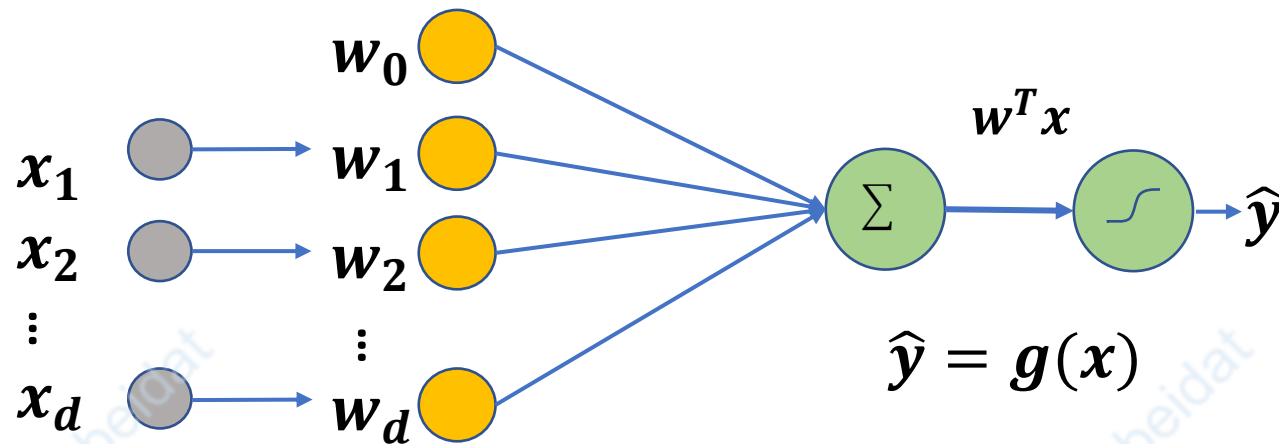
Logistic Classifier

- It is a probabilistic classifier that directly learn $p(Y|X)$ (discriminative).
- It used logistic(sigmoid) function: $\sigma(s) = \frac{1}{1+e^{-s}}$
- Logistic function maps any real value a value between 0 and 1.



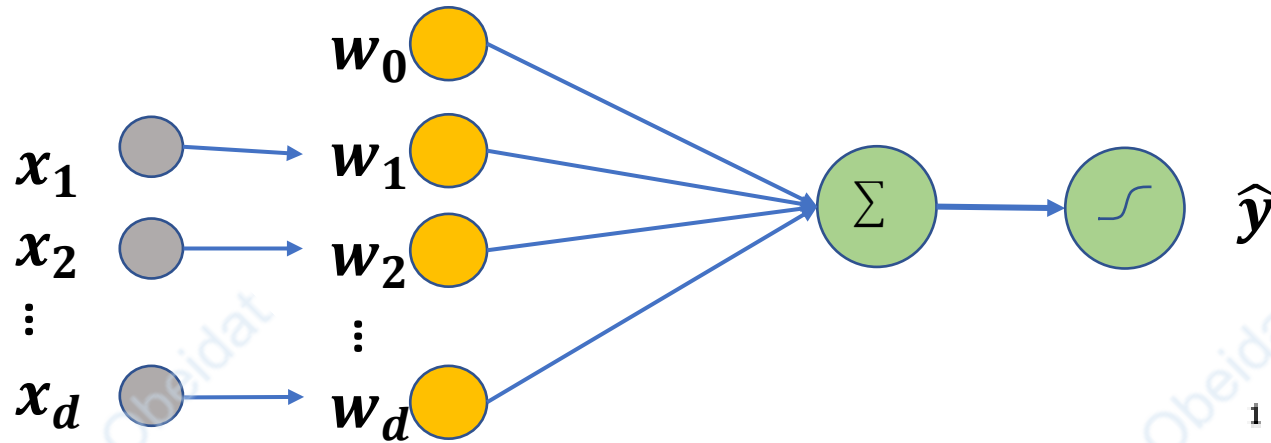
Logistic Classifier

- We assume that we have two classes
- $p(y|x; w)$ is a parametric form of $p(y|x)$ where w is the weight vector (parameters) to be learned.
- $P(y|x; w) = \begin{cases} g(x), & \text{for } y = 1; \\ 1 - g(x), & \text{for } y = 0. \end{cases}$ (formula for likelihood)
- $g(x) = \sigma(w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$



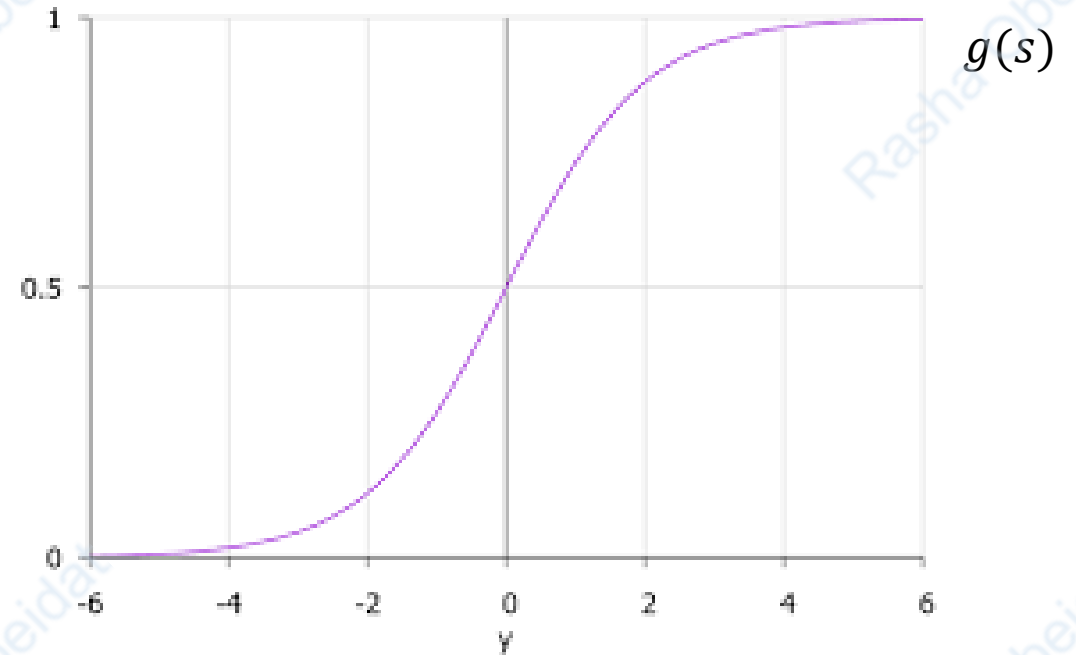
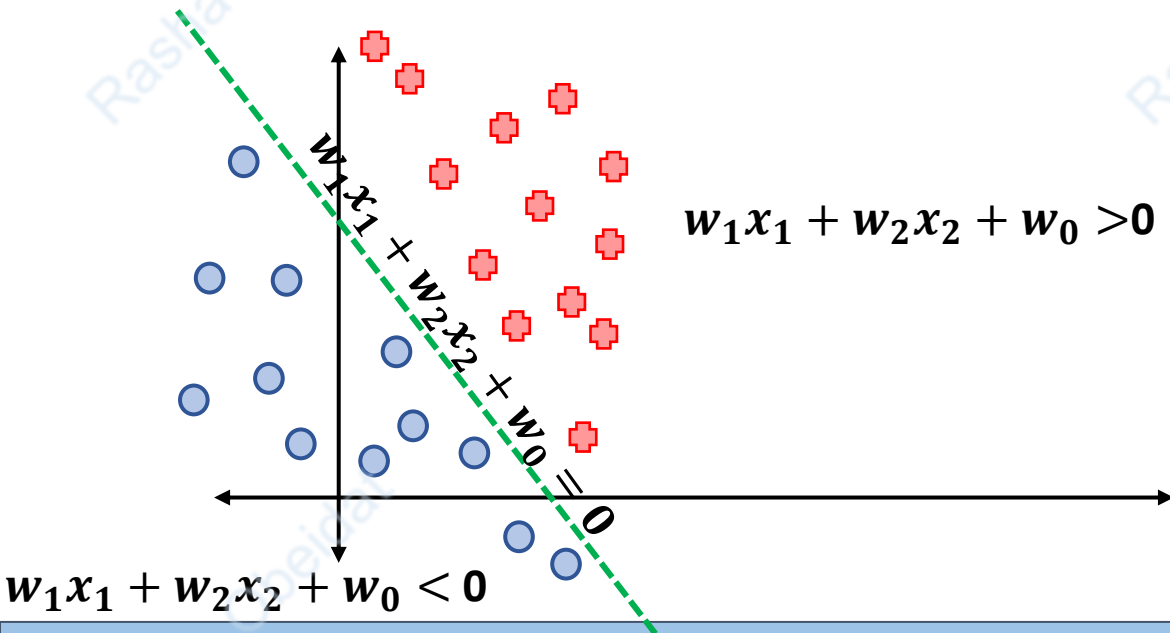
$g(x)$ is interpreted as a probability. The value of $w^T x \in [-\infty, \infty]$, the function g transforms it to a probability.

Logistic Classifier



$g(x)$ is interpreted as a probability. The value of $w^T x \in [-\infty, \infty]$, the function g transforms it to a probability.

$$\hat{y} = g(x) = \sigma(w^T x)$$



Logistic Regression is a Linear Classifier

- Although the **logistic (sigmoid) function** is **non-linear**, logistic regression still produces a **linear decision boundary**.

- **Decision Rule**

- Predict **class 1** if

$$\frac{P(y = 1|x; w)}{P(y = 0|x; w)} > \Theta$$

- For simplicity, let $\Theta = 1$
 - This means we predict class 1 when $P(y = 1|x; w) > P(y = 0|x; w)$

Log-odds

- **Log-odds** is the logarithm of the odds of an event.
- **Odds** measure **how likely** an event is to happen compared to it not happening.
- To compute the **odds** of an event with probability p :

$$\text{odds} = \frac{p}{1 - p}$$

- Example: If $p = 0.8$

$$\text{odds} = \frac{0.8}{0.2} = 4$$

- This means the event is **4 times more likely to happen than not**.

Logistic Regression is a Linear Classifier

- **Log-Odds**
- The **log-odds** of class 1 is defined as

$$\log \frac{P(y = 1|x; w)}{P(y = 0|x; w)}$$

- For logistic regression we obtain

$$\log \frac{P(y = 1|x; w)}{P(y = 0|x; w)} = w^T x$$

- **Key Result**
 - The **log-odds** is a **linear function** of the input features x .
 - Therefore Logistic regression models **linear decision boundaries** Even though the **sigmoid function is non-linear**

Computing Log-Odds of Logistic Regression

- $P(y; = 1 \mid xw) = \sigma(w^T x) = \frac{1}{1+e^{-w^T x}}$

- $P(y; = 0 \mid xw) = 1 - P(y; = 1 \mid xw) = \frac{e^{-w^T x}}{1+e^{-w^T x}}$

Odds : $\log \frac{P(y = 1 \mid x; w)}{P(y = 0 \mid x; w)} = \frac{\frac{1}{1+e^{-w^T x}}}{\frac{e^{-w^T x}}{1+e^{-w^T x}}} = \frac{1}{e^{-w^T x}} = e^{w^T x}$

- **Log-Odds**

- $\log \frac{P(y; = 1 \mid xw)}{P(y; = 0 \mid xw)} = \log \left(e^{w^T x} \right) = w^T x$

- So $\log \frac{P(y; = 1 \mid xw)}{P(y; = 0 \mid xw)} = w^T x$

Learning components

- Training data: $(x^1, y^1), (x^2, y^2), \dots (x^n, y^n)$.

$$x^i = \begin{bmatrix} x_1^i \\ x_2^i \\ \vdots \\ x_d^i \end{bmatrix}$$

- Hypothesis: $g(x; w): X \rightarrow Y$
- w is the parameter vector that we want to estimate its value from the data. e.g., in logistic regression:
- $g(x; w) = \sigma(w_0 + w_1 x_1 + \dots + w_d x_d) = \sigma(\sum_{j=1}^d w_j x_j) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$
- Loss (cost) function: function maps decisions to their associated costs (e.g., binary cross-entropy).
- Optimization method: try to find value of w that minimize the loss function (e.g., gradient decent).

Maximum Likelihood Estimation (MLE)

- Maximum likelihood estimation is a method for estimating **unknown parameters** using observed data.
- It finds the parameter values that **maximize the likelihood of the observed data** under the assumed model.

$$L(\theta) = \prod_{i=1}^n p(x_i | \theta)$$
$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

- To simplify optimization, we maximize the **log-likelihood**:

$$\ell(\theta) = \log L(\theta) = \log \prod_{i=1}^n p(x_i | \theta) = \sum_{i=1}^n \log p(x_i | \theta)$$

MLE for Coin Toss

- **Maximum Likelihood Estimation for the Probability of Heads**

- Let the observed sequence be $Y = \{H, T, H, H, T, H, T, T, H, H\}$

- Define

$$x_i = \begin{cases} 1 & \text{if toss } i \text{ is Heads} \\ 0 & \text{if toss } i \text{ is Tails} \end{cases}$$

- Likelihood function:

$$L(p) = \prod_{i=1}^n p^{x_i} (1-p)^{1-x_i}$$

- Log-likelihood:

$$\ell(p) = \sum_{i=1}^n \log(p^{x_i} (1-p)^{1-x_i}) = \sum_{i=1}^n [x_i \log p + (1-x_i) \log(1-p)]$$

- Let $Y = \sum_{i=1}^n x_i$, Then

$$\ell(p) = Y \log p + (n - Y) \log(1 - p)$$

Finding the MLE

- Differentiate the log-likelihood:

$$\frac{d\ell(p)}{dp} = \frac{Y}{p} - \frac{n - Y}{1 - p}$$

- Set derivative to zero:

$$\frac{Y}{p} - \frac{n - Y}{1 - p} = 0$$

- Solving for p :

$$\hat{p} = \frac{Y}{n} = \frac{\sum_{i=1}^n X_i}{n}$$

- The maximum likelihood estimate of p is the **fraction of heads observed in the data**.

Maximum Likelihood function for LR

- **The goal:** find modeling hypothesis g that best explains the data X . Given the data $(x^1, y^1), (x^2, y^2), \dots (x^n, y^n)$, that are **independently and identically distributed (IID)**, use maximum likelihood estimation to estimate the parameter w .
- Data likelihood:

$$L(w) = L\left(\prod_{i=1}^n p(x^i, y^i; w)\right)$$

The log-likelihood function is:

$$l(w) = \log\left(\prod_{i=1}^n p(x^i, y^i; w)\right)$$

Maximum Likelihood function

$$l(w) = \log (\prod_{i=1}^n p(y^i|x^i; w) p(x^i; w))$$

$$= \sum_{i=1}^n \log(p(y^i|x^i; w)) + \log p(x^i; w)$$

- Since $p(x^i; w)$ doesn't depend on w , it can be dropped (i.e., it is a constant).
- The maximum conditional likelihood estimation:

$$l(w) = \operatorname{argmax}_w \left(\sum_{i=1}^n \log p(y^i|x^i; w) \right)$$

Computing the Likelihood function

- $p(y^i|x^i; w)$ can be written compactly as:
 - $p(y^i|x^i; w) = p(y^i = 1|x^i; w)^{y^i} p(y^i = 0|x^i; w)^{1-y^i}$
- $\sum_{i=1}^n \log p(y^i|x^i; w) = \sum_{i=1}^n \log(p(y^i = 1|x^i; w)^{y^i} p(y^i = 0|x^i; w)^{1-y^i})$
 $= \sum_{i=1}^n y^i \log p(y^i = 1|x^i; w) + (1 - y^i) \log p(y^i = 0|x^i; w)$ (negative of cross-entropy)
- Since $\hat{y}^i = p(y^i = 1|x^i; w)$, $l(w)$ can be written as :
$$l(w) = \sum_{i=1}^n (y^i \log \hat{y}^i + (1 - y^i) \log (1 - \hat{y}^i))$$

Cross-entropy loss

- Maximizing the log likelihood $l(w)$ is **equivalent** to minimizing the binary cross-entropy which is **convex**:

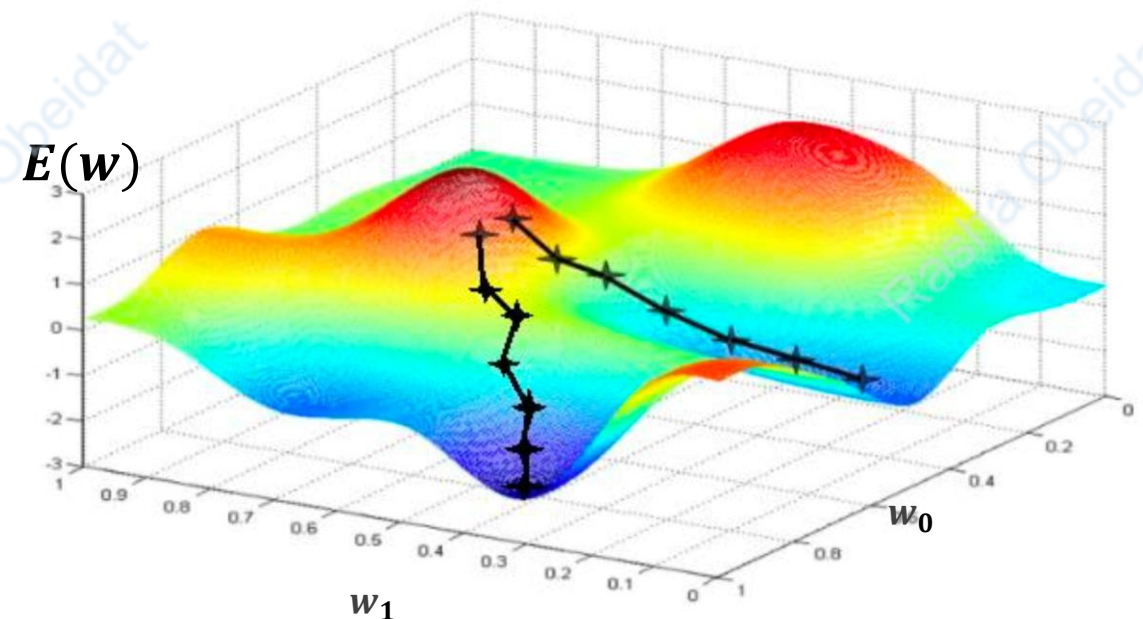
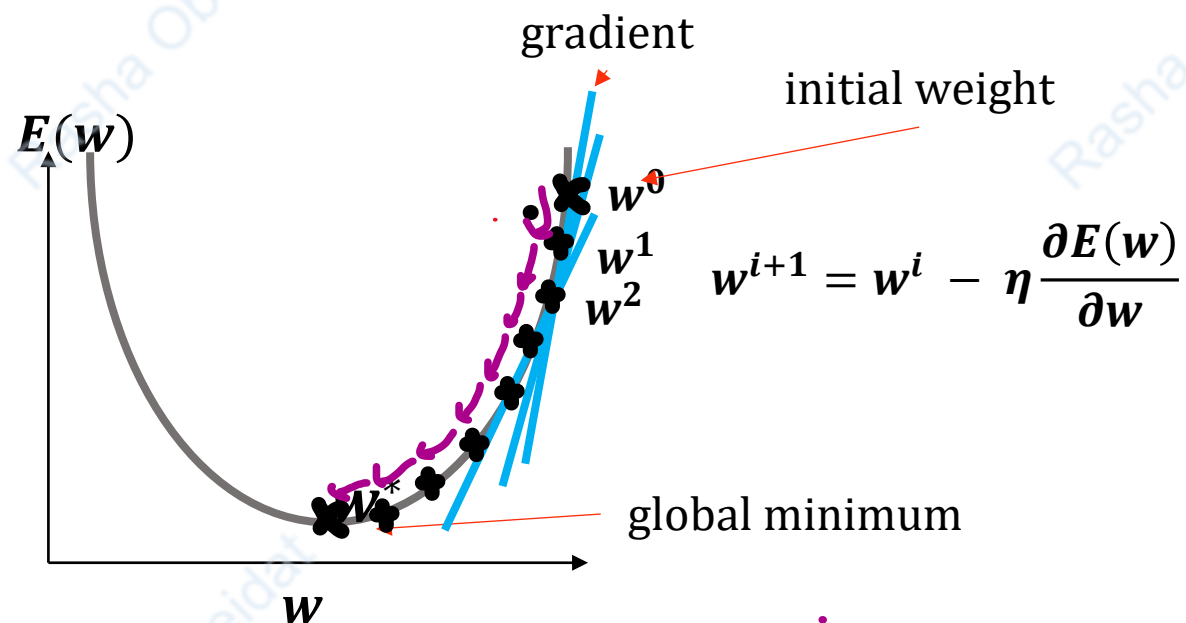
$$E(w) = - \sum_{i=1}^n (y^i \log \hat{y}^i + (1 - y^i) \log(1 - \hat{y}^i))$$

- We can learn the value of w that minimizes the loss function $E(w)$ using **gradient decent**.

$$w^* = \underset{w}{\operatorname{argmin}} E(w)$$

Gradient decent

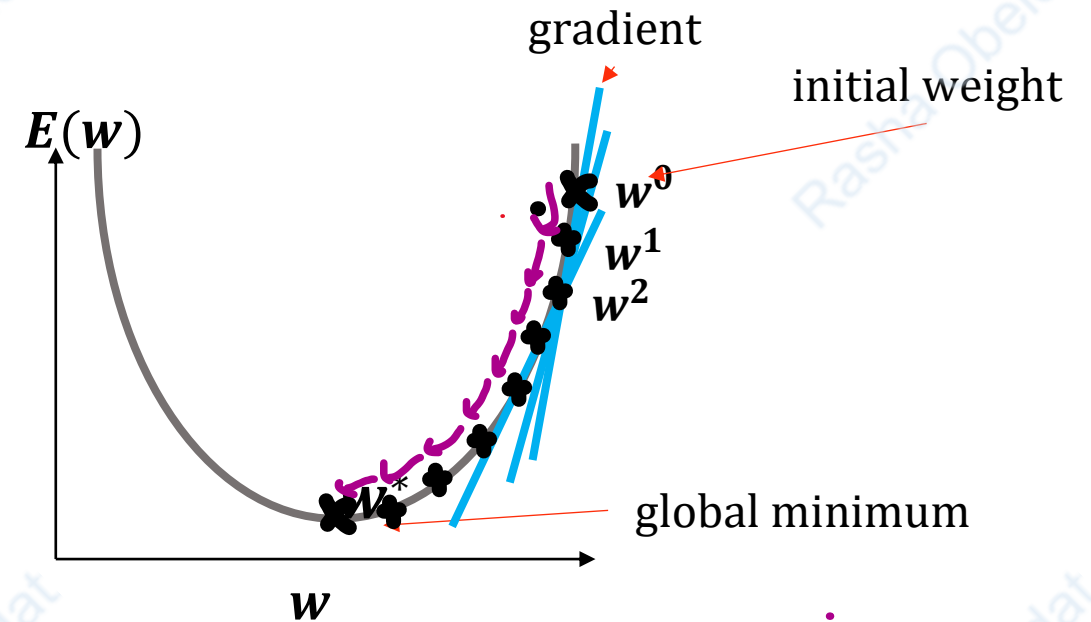
- Start with some random guess of the parameter.
- Iteratively improve the parameter by following the **steepest descent direction** specified by the **gradient of $l(w)$** ($\nabla l(w)$) at the current point.



Gradient decent

- **Finding the Minimum:** To minimize the loss function, we want to update the parameters in the **opposite direction** of the gradient. This is because:
 - A positive gradient indicates that increasing the parameters will increase the loss, so we need to decrease the parameters.
 - A negative gradient indicates that decreasing the parameters will decrease the loss, so we need to increase the parameters.

$$w^{i+1} = w^i - \eta \frac{\partial E(w)}{\partial w}$$



Gradient descent for logistic regression

$$\begin{aligned} E(w) &= - \sum_{i=1}^n y^i \log p(y^i = 1|x^i; w) + (1 - y^i) \log p(y^i = 0|x^i; w) \\ &= - \sum_{i=1}^n (y^i \log p(y^i = 1|x^i; w) + \log p(y^i = 0|x^i; w) - y^i \log p(y^i = 0|x^i; w)) \\ &= - \sum_{i=1}^n (y^i (\log p(y^i = 1|x^i; w) - \log p(y^i = 0|x^i; w)) + \log p(y^i = 0|x^i; w)) \\ &= - \sum_{i=1}^n (y^i \log \frac{p(y^i = 1|x^i; w)}{p(y^i = 0|x^i; w)} + \log p(y^i = 0|x^i; w)) \\ &= - \sum_{i=1}^n (y^i \log \frac{1}{e^{-w^T x^i}} + \log p(y^i = 0|x^i; w)) \\ &= - \sum_{i=1}^n (y^i w^T x^i + \log \frac{e^{-w^T x^i}}{1 + e^{-w^T x^i}}) \\ &= - \sum_{i=1}^n y^i w^T x^i + \log \left(\frac{1 + e^{-w^T x^i}}{e^{-w^T x^i}} \right)^{-1} \\ &= - \sum_{i=1}^n (y^i w^T x^i - \log(1 + e^{w^T x^i})) \end{aligned}$$

Computing the Gradient

$$E(w) = - \sum_{i=1}^n \left(y^i w^T x^i - \log(1 + e^{w^T x^i}) \right)$$

$$\frac{\partial E}{\partial w} = - \sum_{i=1}^n \left(y^i x^i - \frac{x^i e^{w^T x^i}}{1 + e^{w^T x^i}} \right), \quad \text{hint : multiply by } e^{-w^T x^i} / e^{-w^T x^i}$$

$$= - \sum_{i=1}^n \left(y^i x^i - \frac{x^i}{1 + e^{-w^T x^i}} \right) = - \sum_{i=1}^n \left(y^i - \frac{1}{1 + e^{-w^T x^i}} \right) x^i$$

$$= - \sum_{i=1}^n (y^i - p(y^i | x^i; w)) x^i$$

$$\frac{\partial E}{\partial w} = \sum_{i=1}^n (p(y^i | x^i; w) - y^i) x^i$$

$$\frac{\partial E^i}{\partial w} = (p(y^i | x^i; w) - y^i) x^i$$

$$\frac{\partial E}{\partial w_j} = \sum_{i=1}^n (p(y^i | x_j^i; w_j) - y^i) x_j^i$$

Optimizing LR Using Gradient Descent (GD)

- **Our goal** : using **Gradient Descent** to learn the value of w that minimizes the loss function

$$E(w) \quad w^* = \underset{w}{\operatorname{argmin}} E(w)$$

- **How Do We Compute the Gradient?**
- We have **three options**, depending on **how much data we use** at each update:
 - All data at once → Batch Gradient Descent
 - One example at a time → Stochastic Gradient Descent
 - Small groups (mini-batches) → Mini-Batch Gradient Descent
- **Key Idea**
 - All three methods can be used alternatively to optimize the *same logistic regression model*.
 - They only **differ in how often and how much they update the weights**.

Batch GD for Logistic Regression

- **Key Idea:**

Uses **all training examples together** to compute one weight update per iteration.

- **Workflow:**

- Compute predictions for **all samples** in the dataset.
- Compute the total gradient over the full dataset.
- Update the weight vector **once** per iteration.
- Produces **smooth and stable** convergence.

- **Pros:**

- Stable learning updates.
- Converges smoothly when data is clean and small enough.

- **Cons:**

- Requires loading the full dataset (or large chunks) into memory (RAM), which is inefficient or impossible for very large datasets.
- Computationally slow for large datasets.

- **Formula**

- $$W \leftarrow W - \eta \frac{\partial E}{\partial W}$$

$$\frac{\partial E}{\partial w} = \sum_{i=1}^n (p(y^i | x^i; w) - y^i) x^i$$

Batch GD for Logistic Regression

- Given training examples $(x^1, y^1), (x^2, y^2), \dots (x^n, y^n)$.

- Let $w \leftarrow [0 \ 0 \ 0 \ \dots \ 0 \ 0]$

- Repeat for m iterations :

- $d \leftarrow [0 \ 0 \ 0 \ \dots \ 0 \ 0]$

- For $i = 1$ to n :

$$\hat{y}^i = \frac{1}{1 + e^{-w^T x^i}}$$

$$\text{error} = \hat{y}^i - y^i$$

$$d = d + \text{error} \cdot x^i$$

- $w \leftarrow w - \eta d$

Or Repeat until $|d| < \epsilon$

, ϵ : convergence criterion, $|d|$ is $|\nabla E(w)|$

η is the learning
rate (step size)

Stochastic (online) GD

- **Key Idea:**

Updates the weights **after each individual training example**, rather than waiting for the whole dataset.

- **Workflow :**

- For each sample (x^i, y^i) , compute prediction and update weights immediately.
- Updates are **faster** and can start learning right away.
- The path of learning is **noisy** (jumps around) but may escape poor local minima.

- **Pros:**

- Very fast updates.
- Scales to massive datasets (data streams).
- Can improve generalization by introducing noise.

- **Cons:**

- Updates are noisy $\rightarrow$ convergence may oscillate.
- Needs tuning of learning rate to avoid divergence.

- **Update Rule:**

$$w \leftarrow w - \eta(\hat{y}^i - y^i)x^i$$

Stochastic (online) GD

- Given training examples $(x^1, y^1), (x^2, y^2), \dots (x^n, y^n)$.

- Let $w \leftarrow [0 \ 0 \ 0 \ \dots \ 0 \ 0]$

- Repeat for m iterations

- For $i = 1$ to n :

$$\hat{y}^i = \frac{1}{1 + e^{-w^T x^i}}$$

$$\text{error} = \hat{y}^i - y^i$$

$$w \leftarrow w - \eta(\text{error} \cdot x^i)$$

Mini-Batch GD (Practical Standard)

- **Key Idea:**

- Splits data into **small batches** (e.g., 32, 64, 128 samples) and updates weights per batch.
- Compute partial gradient using a **subset** of the data.
- Mini-batch size controls the **trade-off** between efficiency and stability.
- Used in most deep learning frameworks (TensorFlow, PyTorch).

- **Workflow :**

- Shuffle and split the training dataset into small batches (e.g., 32, 64, 128 samples each).
- For each batch: compute predictions, calculate the error, and compute the gradient using only that batch.
- Update the weights using the batch gradient, then move to the next batch until all batches are processed (one epoch in NN, iteration in LG).

- **Benefits:**

- Faster than batch GD.
- More stable than SGD.
- Can be optimized using GPUs (matrix operations on batches).

- **Typical Setting:** Batch size $\in \{16, 32, 64, 128\}$

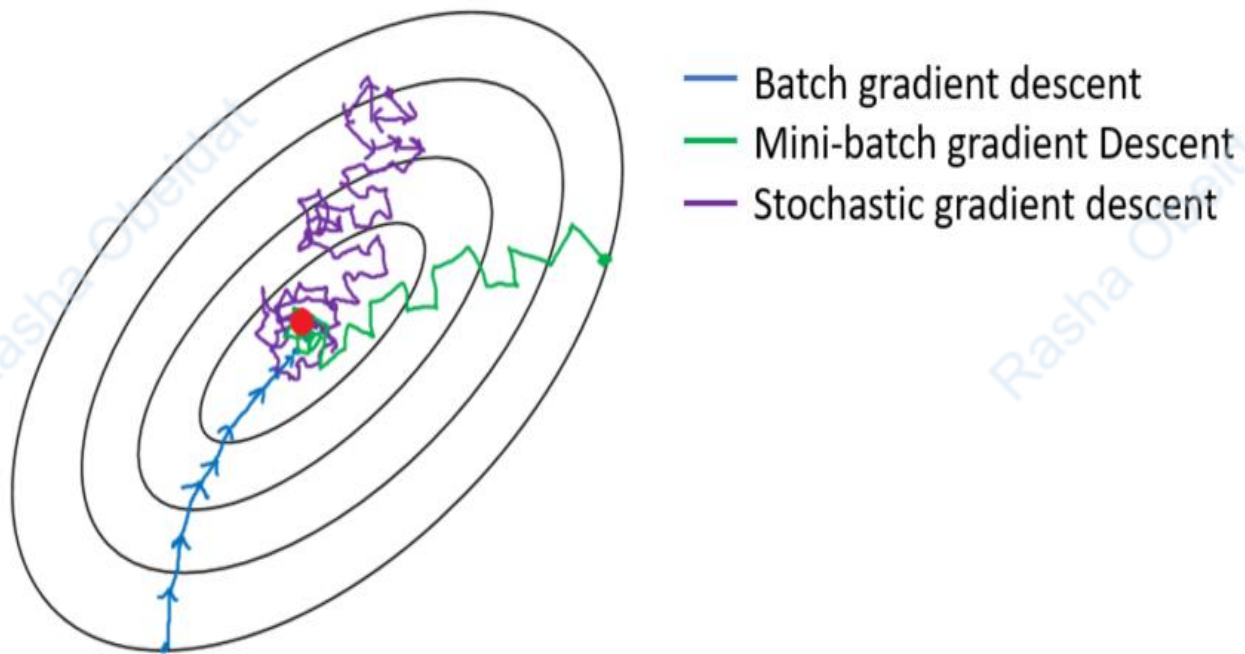
- **Update Rule:**

$$w \leftarrow w - \eta \cdot \sum_{i=1}^b (\hat{y}^i - y^i) x^i$$

Mini-batch GD

- Given training examples $D_t = \{(x^1, y^1), (x^2, y^2), \dots (x^n, y^n)\}$.
- Let $w \leftarrow [0 \ 0 \ 0 \ \dots \ 0 \ 0]$
- Repeat Repeat for m iterations :
 - Repeat until the n training examples are used:
 - Sample b examples D_t (without replacement)
 - $d \leftarrow [0 \ 0 \ 0 \ \dots \ 0 \ 0]$
 - For $i = 1$ to b :
$$\hat{y}^i = \frac{1}{1 + e^{-w^T x^i}}$$
$$\text{error} = \hat{y}^i - y^i$$
$$d = d + \text{error} \cdot x^i$$
 - $w \leftarrow w - \eta d$

Gradient Descent Trajectories



[2]: Gradient descent variants' trajectory towards minimum

Gradient Descent Trajectories

- **Batch Gradient Descent**

- Produces a **smooth and stable trajectory**.
- Each step follows the **true gradient direction**.
- Progress is steady but may require **large computations per step**.

- **Stochastic Gradient Descent**

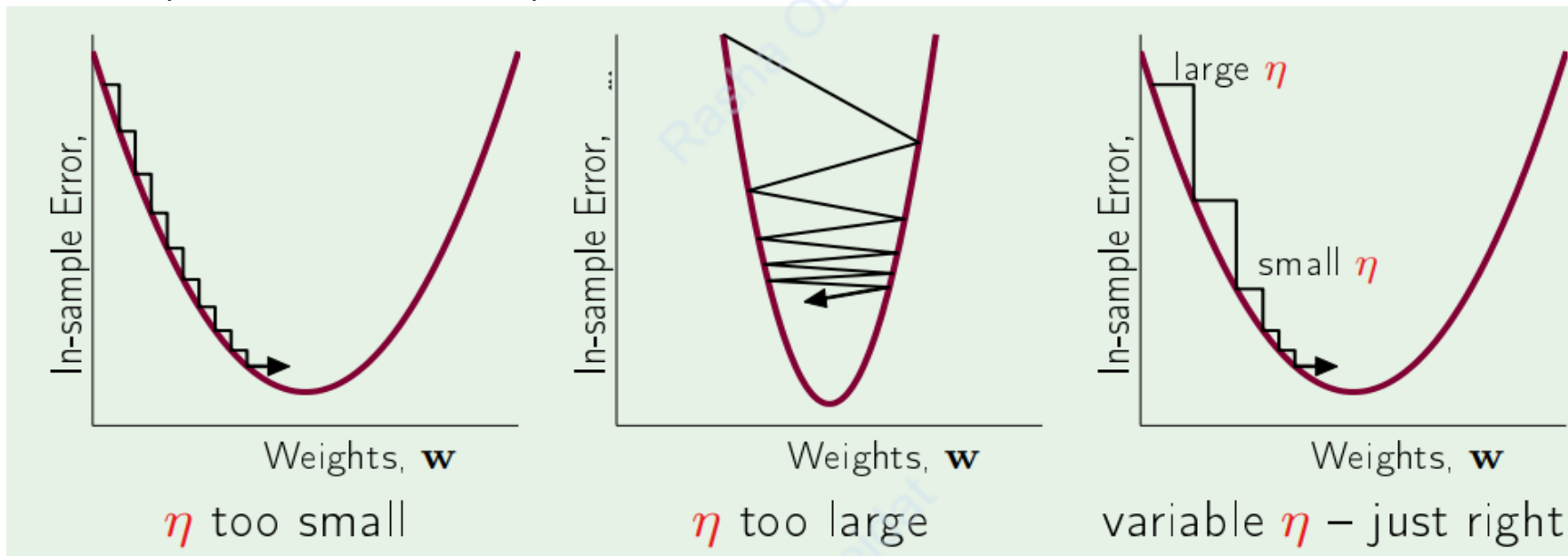
- Trajectory is **highly noisy and zig-zagging**.
- Each update uses only one sample, so the gradient estimate is **very noisy**.
- May **oscillate around the minimum** instead of converging smoothly.

- **Mini-Batch Gradient Descent**

- Path is **less noisy than SGD but less smooth than batch GD**.
- Small fluctuations due to **approximate gradients**.
- Typically **converges faster while remaining relatively stable**.

Effect of learning rate (η) on GD

- Large η progresses faster but may lead to oscillation.
- Small η avoid overstepping but converges slower.
- Adaptive Learning rate (aka Learning rate scheduling): dropping the learning rate by a step decay every m **epochs**, where an **epoch** is going through the whole training set once. (e.g., $\frac{1}{t^2}$)
- More advanced techniques Adaptive gradient that uses different rate for different dimensions (ADAM, AdaGrad...)



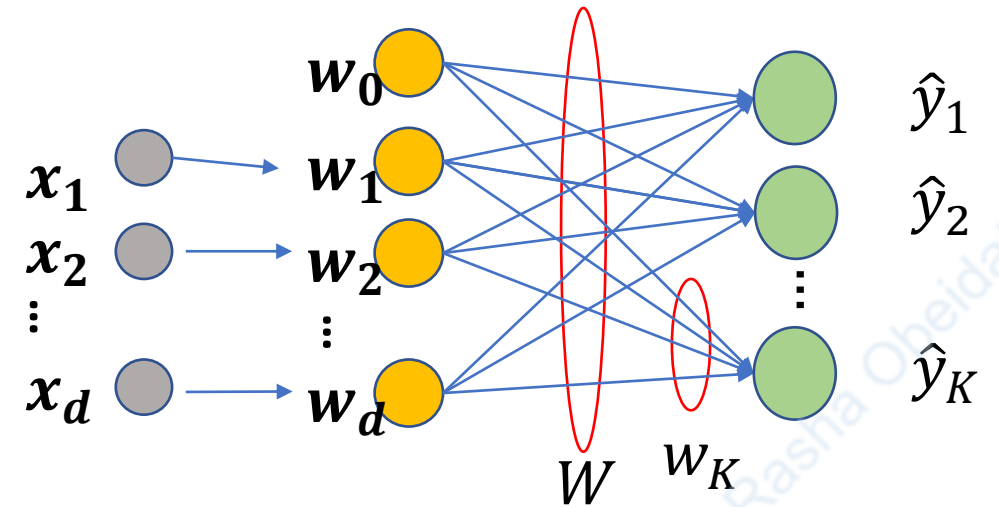
Multi-class logistic regression

- If we have more than 2 classes, we have two approaches
 1. Train several binary classifier and combine their output: one-vs-rest and one-vs-rest.
 2. Soft-max function

Multi-class logistic regression

- For $K > 2$ classes, we can define the posterior probability using the soft-max function:

$$p(y = k|x; w_k) = \hat{y}_k = \frac{e^{w_k^T x}}{\sum_{j=1}^K e^{w_j^T x}}$$



Going through a similar MLE derivations, we get the following gradient:

$$\nabla E(w_k) = \sum_{i=1}^n (\hat{y}_k^i - y_k^i) x^i$$

Where $y_k^i = 1$ if $y^i = k$, and 0 otherwise for $k = 1, \dots, K$.

Normalization: Motivational example

- **Housing Example** : Suppose we want to train **Logistic Regression** to classify houses into:
 - Desirable
 - Undesirable
- Using two features:

Feature	Typical Range	Scale Difference
Number of rooms	1 to 6	Small range
House area (m ²)	50 to 300	Large range

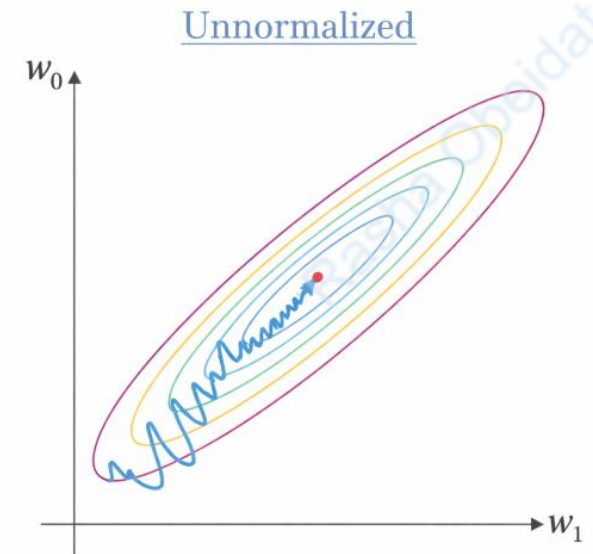
- Because **house area** has values much larger in magnitude than **number of rooms**, it will **dominate the gradient**.

This means the model will mainly use **area** to make decisions and largely **ignore the number of rooms**, even if rooms are as important as area.

Normalization: Motivational example

- **How This Affects Logistic Regression**

- To compensate for this imbalance, Logistic Regression will try to **shrink the weight** of the large-scale feature (house area) and **increase the weight** of the small-scale feature (number of rooms). But this compensation is **not perfect**.
- The optimization landscape becomes **elongated / skewed** (contours look like stretched **ovals** rather than circles).
- Gradient descent takes **zig-zag steps**, moving slowly toward the minimum (large η are not stable).
- Training becomes **slower and less stable**.



Normalization

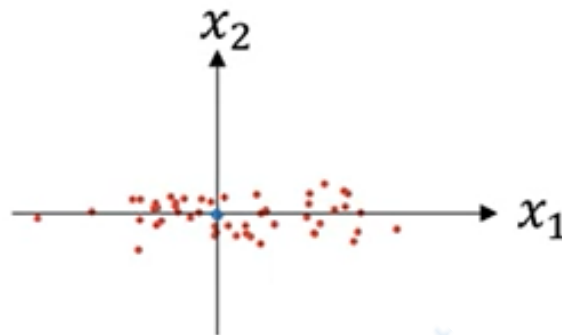
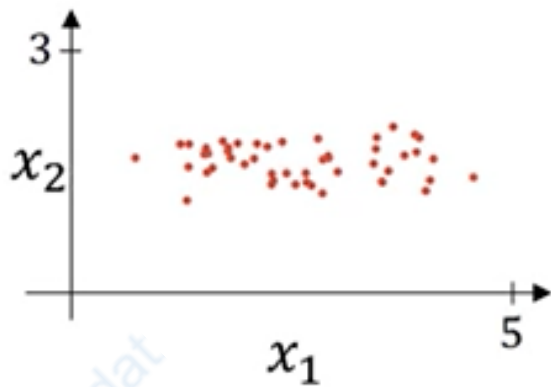
- **Min-Max feature scaling:** $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$

Scale all features to values between 0 and 1

- **Standardization :** $x' = \frac{x - \mu}{\sigma}$

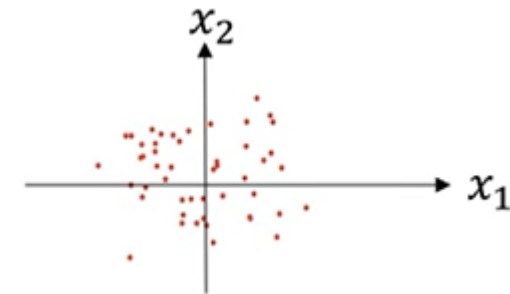
- set the mean of x to 0 and σ to 1.

- Use the μ , σ you computed from the training data to normalize the testing and the dev data. Same thing for x_{min} , x_{max} .



By subtracting μ

Standardization



By dividing by σ

Why is normalization important?

- **Feature Scaling:**

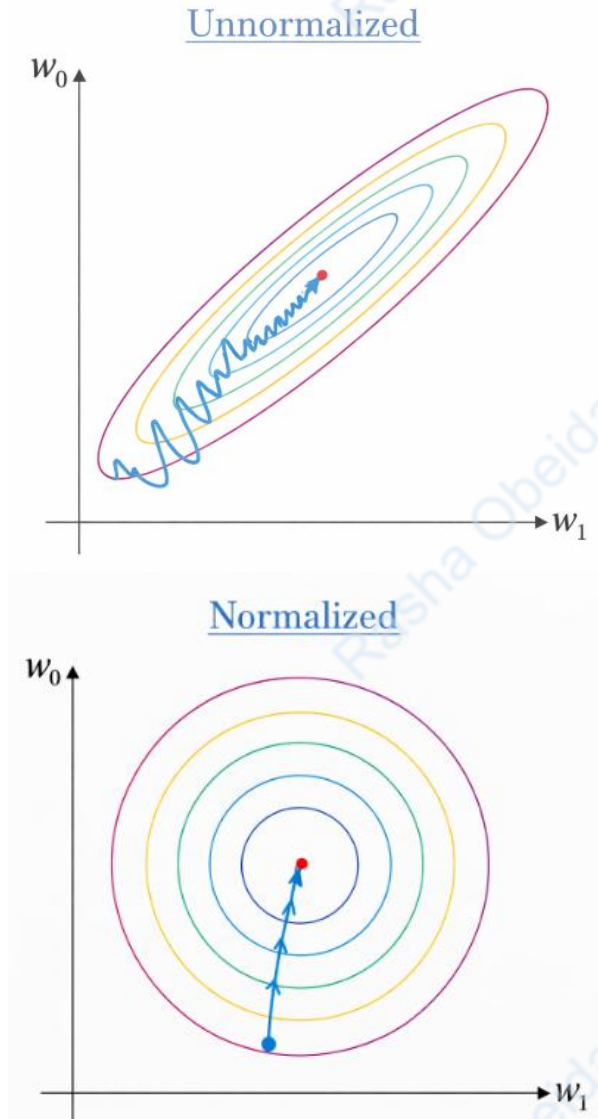
Puts features on the **same scale**, preventing the model from being biased toward features with larger numeric values.

- **Improved Convergence:**

Makes the optimization surface **more round and smooth**, so gradient descent moves **more directly** toward the minimum → **faster and more stable learning**.

- **Better Interpretability:**

When features are similarly scaled, it's **clearer how each feature influences the prediction**, since weights are now more comparable.



Normalization Must be Performed Based on the Training Data Only

- When we normalize features, we compute statistics such as:
 - **Min and Max** (for Min-Max scaling), or
 - **Mean μ and Standard Deviation σ** (for Standardization)
- These values **must be computed using only the training data**.
- **Why?**

If we compute scaling values separately for the validation or test set:

- We **leak information** about those sets into the model training.
- The model would be seeing data it should **never have access to** during training
- This leads to **unfair evaluation and over-fitting**.

Normalization: Correct Workflow

- **Fit** the scaler on the **training data only** (compute μ , σ or min, max).
- **Apply (transform)** the same scaler to:
 - Training data
 - Validation (dev) data
 - Test data
- **Do not re-fit** on validation or test data.

Normalization: Correct Workflow

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
# Step 1: Fit on training data only
scaler.fit(X_train)
# Step 2: Apply the same scaling to train, dev, and test
X_train_scaled = scaler.transform(X_train)
X_dev_scaled = scaler.transform(X_dev)
X_test_scaled = scaler.transform(X_test)
```



```
scaler1 = StandardScaler()
scaler1.fit(X_train)
X_train_scaled = scaler1.transform(X_train)
scaler2 = StandardScaler()
scaler2.fit(X_dev)
X_dev_scaled = scaler2.transform(X_dev)
scaler3 = StandardScaler()
scaler3.fit(X_test)
X_test_scaled = scaler3.transform(X_test)
```



```
Data = np.vstack([X_train, X_dev, X_test])
scaler = StandardScaler()
scaler.fit(Data)
X_train_scaled = scaler.transform(X_train)
X_dev_scaled = scaler.transform(X_dev)
X_test_scaled = scaler.transform(X_test)
```



Why code causes data leakage ?

Regularization

- Helps creating a more robust model , effective at making predictions on new data
- Balance the trade-off between fitting the training data well and keeping the model simple.
- For logistic regression

Regularization term

$$w^* = \underset{w}{\operatorname{argmin}} E(w) + \frac{\lambda}{2} \sum_j w_j^2$$

$$\frac{\partial E^i}{\partial w} = (p(y^i | x^i; w) - y^i) x^i$$



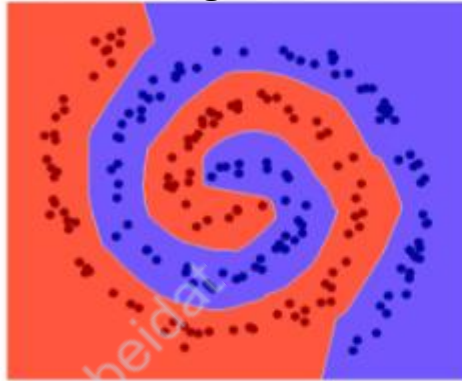
$$\frac{\partial E^i}{\partial w} = (p(y^i | x^i; w) - y^i) x^i + \lambda w$$

Regularization

- In regularization, λ is used in the loss function to impose a penalty on the size of the model's coefficients (weights). The general form of the regularized loss function.
- **Controlling Complexity:**
 - **Large λ** : A high value of λ increases the penalty on the weights, leading to a simpler model with potentially fewer features being selected or focused on
 - **Small λ** : A lower value reduces the penalty, allowing the model to fit the training data more closely. However, this increases the risk of overfitting.

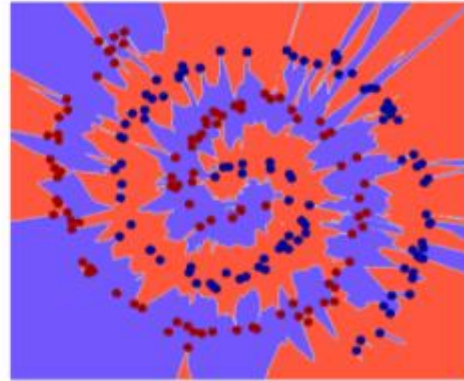
Regularization

with Regularization

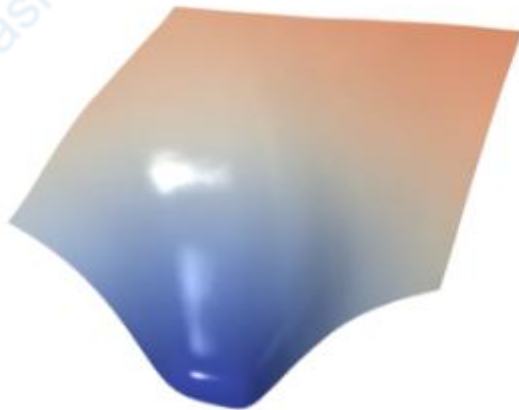


(a) 100% train, 100% test

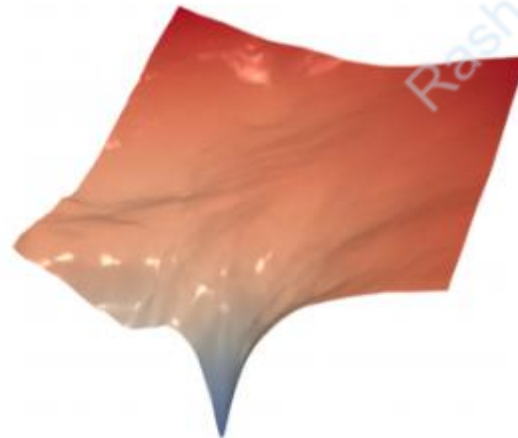
without Regularization



(b) 100% train, 7% test

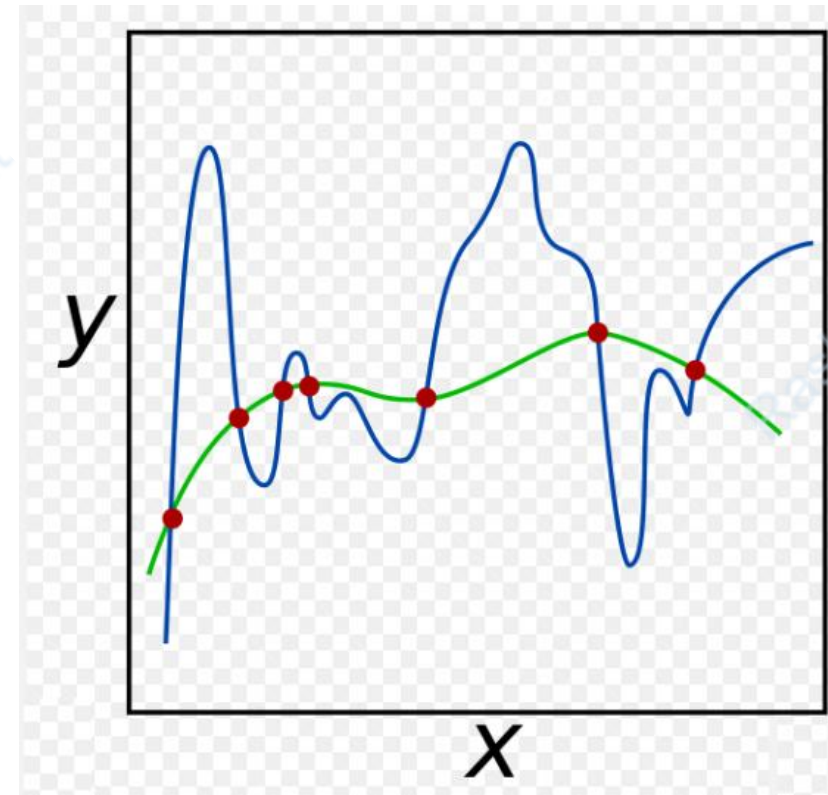


(c) Minimizer of network in (a) above



(d) Minimizer of network in (b) above

Blue: without Regularization
green: with Regularization



Huang, W. Ronny, et al. "The Blessing of Dimensionality: An Empirical Study of Generalization."

Model Training

- **Key Steps in Training a Model:**

- **Data Preparation:** Ensure data is clean, formatted correctly, and split into features (X) and labels (Y).
- **Selection of a Training algorithm:** Choose an appropriate algorithm based on the problem type (e.g., regression, classification, deep learning model, machine learning model).
- **Learning Process:** Use algorithms to *iteratively* adjust parameters, typically by minimizing a loss function.
- **Monitoring Performance:** Employ metrics (training and validation accuracy or loss) to monitor the model's learning progress.

These slides are the intellectual property of **Dr. Rasha Obeidat®** .

They may be used for teaching this course but may not be
modified or redistributed without permission

Contact: Obeidat.rm@gmail.com

Rmobeidat@just.edu.jo

